

# Contents

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Home</b>                                                             | <b>2</b>  |
| <b>Algorithm Overview</b>                                               | <b>3</b>  |
| What is a container? . . . . .                                          | 4         |
| Running the code on linux-based (and MacOS) machines . . . . .          | 4         |
| Running the code on Windows machines . . . . .                          | 6         |
| Running the code on HPC machines using Singularity . . . . .            | 6         |
| <b>Controlling the execution of the retrieval</b>                       | <b>6</b>  |
| Command line arguments . . . . .                                        | 6         |
| VIP file – high level overview . . . . .                                | 7         |
| <b>VIP File (operational processing)</b>                                | <b>7</b>  |
| <b>Overview of typical input observations</b>                           | <b>9</b>  |
| Infrared spectrometer . . . . .                                         | 9         |
| Basic instrument / measurements . . . . .                               | 9         |
| The IRS block in the VIP file . . . . .                                 | 10        |
| Noise filtering . . . . .                                               | 11        |
| Microwave radiometer . . . . .                                          | 12        |
| Basic instrument / measurements . . . . .                               | 12        |
| The MWR block in the VIP file . . . . .                                 | 13        |
| MWRscan_type (describe each, including format) . . . . .                | 13        |
| Bias determination / correction . . . . .                               | 14        |
| Surface met . . . . .                                                   | 14        |
| Basic instrument / measurements . . . . .                               | 15        |
| ext_sfc_XX_type (describe each) . . . . .                               | 15        |
| Assumed uncertainties . . . . .                                         | 15        |
| Ceilometer . . . . .                                                    | 15        |
| Basic instrument / measurement . . . . .                                | 16        |
| Possible formats . . . . .                                              | 16        |
| <b>Importance of the prior information</b>                              | <b>16</b> |
| Thermodynamic prior covariance matrix (external to code) . . . . .      | 16        |
| Vertical grid . . . . .                                                 | 17        |
| Prior recentering . . . . .                                             | 17        |
| Specifying the prior . . . . .                                          | 18        |
| Cloud prior covariance information . . . . .                            | 18        |
| Trace gas prior covariance . . . . .                                    | 19        |
| Actual retrieved fields . . . . .                                       | 21        |
| Temperature and humidity . . . . .                                      | 21        |
| Cloud properties . . . . .                                              | 21        |
| Uncertainties in the actual retrieved fields . . . . .                  | 21        |
| Information content in actual retrieved fields . . . . .                | 22        |
| Derived fields . . . . .                                                | 22        |
| Profiles . . . . .                                                      | 22        |
| Indices . . . . .                                                       | 22        |
| Understanding obs_flag, obs_dim, obs_vector, and forward_calc . . . . . | 23        |

# Home

Thermodynamic profiles are considered critical for a huge range of both operational and research objectives (Wulfmeyer et al. 2015). In particular, there is a huge need for accurate high temporal resolution temperature and humidity profiles in the planetary boundary layer (PBL), where synoptic boundaries, mesoscale processes and circulations, the diurnal cycle, and much more can modulate these profiles. These profiles are used for process-study research over a range of topics spanning mountain meteorology to turbulent mixing to moist cloud processes, are critically needed to evaluate numerical weather forecasts and serve as input via data assimilation to these models, and for a large number of downstream stakeholders such as the renewable energy community. This massive need served as the motivating force behind the development of this algorithm.

The Tropospheric Remotely Observed Profiling via Optimal Estimation (TROPOe) software product is designed to retrieve thermodynamic profiles and cloud properties from ground-based remote sensors. It is a robust retrieval system that is able to use a wide number of remote sensing and in-situ observations as input to provide high temporal resolution temperature and humidity profiles. The primary input data streams are made by an infrared spectrometer (IRS) or a multi-channel microwave radiometer (MWR), but the algorithm can include water vapor and temperature observations from lidars, radio acoustic sounding systems, radiosondes, surface met stations, numerical weather prediction model output, and more. The observations are constrained by an a priori climatology to provide realistic solutions. A complete characterization of the uncertainties of the retrieved profiles, as well as the information content in those profiles that comes from the observations, are included in the output files created by the algorithm.

The research and development activities that led to TROPOe was also inspired by the Department of Energy Atmospheric Radiation Measurement (ARM) program (Turner and Ellingson, 2016). In particular, ARM-funded scientists developed and matured the Atmospheric Emitted Radiance Interferometer (AERI; Knuteson et al 2004a, 2004b). The primary goals of the AERI were multiple (Turner et al. 2016), but one of them was to provide high temporal resolution water vapor and temperature profiles in the atmospheric boundary layer. In the mid-1990s, W.L. Smith and his colleagues at the University of Wisconsin – Madison developed the AERIprof algorithm to retrieve these profiles (Smith et al. 1999), and used it to perform a range of research studies (e.g., Feltz et al. 1998; Feltz et al. 2003; Wagner et al. 2008). There were multiple shortcomings with AERIprof: (1) it needed a very good first guess in order to converge to a solution; (2) it was primarily limited to clear-sky scenes, as it is difficult to get good estimates of cloud properties to use in the first guess; (3) it did not provide an uncertainty estimates for its retrieved profiles; (4) it was a physical-iterative retrieval that used a statistical fast radiative transfer model that was trained using ARM Southern Great Plains (SGP; Sisterson et al. 2016) observations and could not be run at other locations; and (5) it used a fixed value for the carbon dioxide concentration in this statistical radiative transfer model.

TROPOe has been in active development since 2012 by D.D. Turner. At that time, the original motivation was to address the shortcomings in AERIprof by using a Bayesian physical-iterative approach to get accurate retrieved profiles with their uncertainties; indeed, the original algorithm was called AERIOe. However, there is a large community using ground-based MWRs for thermodynamic profiling, which led to the desire to use the same approach for both infrared spectrometers (IRS) and microwave radiometers (MWR). Thus, TROPOe evolved to use either dataset, or both simultaneously, as input.

Retrieving thermodynamic profiles from passive remote sensors like IRS and MWR is non-trivial. It is an ill-posed mathematical problem, and could be addressed either with a purely statistical retrieval or a physical-iterative method (Maahn et al. 2020). TROPOe uses the latter approach, which requires that a forward model (F) be used to convert the “state space” (i.e., profiles of temperature and humidity, cloud information, etc.;

call this  $x$ ) into the “observation space” (i.e., the radiance observations made by the IRS and MWR; call this  $y$ ). So  $F(x) = y$ . We use radiative transfer models, which require additional inputs, as part of this process. We have encapsulated the retrieval software, radiative transfer models, the ancillary information needed by the radiative transfer models, and more into a software container that eases the distribution and use of this retrieval system.

TROPoe was designed to be both an operational algorithm and a research algorithm. As such, it is both easy to use and yet extremely flexible. To handle the flexibility and thus configure the retrieval, we use a “Variable Input Parameter” or VIP file. The VIP file is configured as a key-value system, and there are over 200 keys that are predefined that can be overridden by the user. The output file contains a wealth of information for each retrieved profile, including uncertainty estimates, measures of the correlated error, the information content, the observations and forward calculation for the converged retrieval, and a range of fields that can be used for quality control. It incorporates years of experience developing and using retrieval algorithms, and decades of improvements in the radiative transfer models that are the underpinnings of the retrieval. All of these components are discussed in subsequent chapters. We have attempted to make this Users Guide as complete as possible to help you configure the retrieval to use the observations you desire, to help you interpret the output, to provide the guidance needed to understand the role of the a priori dataset, and more. Additional information can be found in the references that describe TROPoe (formerly AERIoe). Output from this algorithm has been used in a wide range of studies, resulting in over 30 published papers.

## Algorithm Overview

Physical retrievals exploit the idea, introduced earlier, that observations, forward models, and the assumptions necessary to drive them have inherent uncertainty and can be represented by probability distributions. Suppose we have an observation  $y$ , and desire the atmospheric state  $x$  that existed when the observation was taken. These algorithms typically use Bayes’s theorem:

$$P(x|y) = P(y|x)P(x)/P(y)$$

which states how probable a certain  $x$  is given  $y$  [i.e.,  $P(x|y)$ ]. This probability density function is already the solution for the inverse problem, which means that the solution includes an uncertainty estimate. Bayes’s theorem tells us that it can be obtained by multiplying the probability of  $y$  given  $x$  [ $P(y|x)$ ; the likelihood] with the a priori probability density of the state  $P(x)$  (e.g., based on a climatology, a model forecast, or a measurement at some distance away in space or time). The probability  $P(y|x)$  represents the uncertainty in the measurement process and also in the model that converts the state  $x$  into observations  $y$ , while the division by  $P(y)$  is effectively a normalization factor.

The optimal estimation (OE) is a widely used physical retrieval method and the equations, given by the excellent textbook by Rodgers (2000), can be derived from Eq. 1 if one assumes all of the probabilities have a Gaussian distribution. OE also implicitly assumes that there is no bias in the measurements and prior dataset; in other words, the average of their uncertainties is zero. A natural advantage of OE is that the uncertainties in the retrieved solution (i.e., the uncertainties in  $x$ ) are automatically provided as a full error covariance matrix, which includes the uncertainties of the observations  $y$ , the uncertainties in the a priori information used to constrain the retrieval, and the sensitivity of the forward model ( $F$ ).

A detailed description of TROPoe is given by Turner and Löhnert (2014), which includes equations that specify how the next iteration is derived in this physical-iterative technique, uncertainty estimates, and information content. Turner and Blumberg (2019) indicate how the observation vector  $y$  can be expanded to include observations from multiple instruments. Turner and Löhnert (2021) continue this exploration of using

multiple different observation types in  $y$ , and quantify the information content when different observations are added to  $y$ . All of the results in these papers were generated directly from output in the TROPoe data files; thus, these papers demonstrate how the information content and uncertainties change for different retrieval configurations.

Passive retrievals are, by their nature, ill-posed mathematical problems. In other words, there often exists more than one state space solution that maps to a given observation. Stated mathematically, this implies  $F(x_1) = y$  and  $F(x_2) = y$ , where  $x_1 \neq x_2$ . Furthermore, every observation has some amount of random uncertainty associated with it; i.e.,  $y_{\text{true}} = y + \epsilon$ , where  $y_{\text{true}}$  would be the observation from a perfect instrument. Thus, we need to constrain the solution (i.e., the retrieved  $x$ ) so that the solution is “reasonable”. This is done using a priori information, which is given by  $P(x)$  in Eq 1. Additional information on the a priori will be discussed in a later chapter.

There are a lot of components needed to run TROPoe. These include the retrieval code itself, the radiative transfer models, the input for the radiative transfer models (e.g., the spectroscopic databases and cloud property scattering databases), standard atmospheres, and more. All of these components must be installed in a particular configuration to allow the retrieval to work. This was done on individual computer systems in the past, but would often take a full day to compile the radiative transfer models and get the configuration set up. This was unsustainable. We have moved to using Docker containers to create a packaged software system that can easily be downloaded off the internet.

## What is a container?

A software container is a relatively lightweight executable package of code that contains everything it needs to run. Thus, software in a container can be run in a standalone manner without any significant dependencies that may be difficult to configure. In this example, we use a Docker container, as it is an open-source platform that makes it easy to build, deploy, run, and update. We, the TROPoe developers, build updated container images when we make improvements to the retrieval code, or the radiative transfer models have been updated, or similar improvements. We then push the new container image to DockerHub, which is a web-based platform very similar to GitHub. Users, such as yourselves, pull the new container image to your computer and then run it. This makes updating the codebase extremely easy and robust.

Containers can be run on Linux, Windows, and MacOS environments. One advantage of using containers is that the environment and software is the same for all users and thus two people running the same data with the same version of the container on different computing systems will get the same answer.

## Running the code on linux-based (and MacOS) machines

While the container has the required software environment, the code needs access to the data it will process as well as the a priori data needed to constrain the retrieval (more on the a priori dataset latter). This requires that we “bind” a directory that has our observational data to the container; this process is similar to a network file system (NFS) mount. Additionally, we need to pass in other information such as the date to process, some control parameters, etc. We can do this via the command line on a linux / MacOS operating system.

For a unix-based operating system like a linux server or a MacOS computer, we can use either “docker” or “podman” to get and execute the TROPoe software container image. [Your system administrator may have preferences about which they would like to install / support, but the execution is virtually identical for each. For this discussion, we will assume we are using docker.] To get the latest version of the TROPoe software from DockerHub, issue this command:

```
docker pull davidturner53/tropoe:latest
```

This will pull the software down to your computer. You will see it updating in layers, as that is how it was built. Another important command is to see what software images you have on your computer. This can be done using:

```
docker image ls
```

In the statements above, replace “docker” with “podman”, if your system only supports the latter. Note that there are a multitude of other docker (podman) commands available; a quick online search may be informative for you. Now you are ready to run the TROPoe software package. We have created a bash shell script to make a simple command line interface, and the script takes care of binding the external directories and passing in the arguments as environmental variables. This script, `run_tropoe_ops.sh`, is executed like this:

```
run_tropoe_ops.sh date VIPfile priorFile shour ehour verbose dataDir tmpDir imageName
```

Where:

- `date` is an 8-digit date of the format `yyyymmdd`
- `VIPfile` is the name of the VIP file (details on this later). Note you should enter a relative path starting from `dataDir`
- `priorFile` is the name of the a priori file (details on this later). Again, use a relative path starting from `dataDir`
- `shour` is the starting hour for the retrievals (usually should be 0 – units are UTC)
- `ehour` is the ending hour for the retrievals (maximum is 24 – again, units are UTC). Note that if `ehour`  $\leq$  `shour` then the code will perform a single retrieval at `shour`
- `verbose` dictates how “noisy” the output to the screen is. The values range from 1 (pretty quiet) to 3 (very noisy)
- `dataDir` is the “head” of the data tree where your input observational data, `VIPfile`, and `priorFile` are located.
  - Note that the container will map this directory (suppose it is `/home/user/my_input_data`) to the directory `/data` inside the container. In other words, the file `/home/user/my_input_data/thisfile` will look like `/data/thisfile` from inside the container.
  - Note that the code will expect to find the `VIPfile` using `dataDir/VIPfile`, which is why the `VIPfile` should be a relative path with respect to `dataDir`. The same is true for the `priorFile`. Also, because of the way the directory binding works, the container can only see directories and files that are in `dataDir`. Thus, all of your input data must be in that directory
- `tmpDir` is only used for two purposes: (1) if you have a very fast hard drive that can be used for the input/output files that are created regularly as part of the retrieval process (the radiative transfer models used by TROPoe use temporary files heavily), or (2) you would like to see these temporary files perhaps for debugging purposes. Note that, by default, TROPoe won’t use this `tmpDir`; instead it uses a temporary directory inside the container that it can more easily clean up. Thus, we recommend that you set `tmpDir` to be the same as `dataDir`, or perhaps `dataDir/tmp`.
- `imageName` is the name of the container image you want to run. When a new TROPoe container is built, we tag it two ways: (1) with the date it was built (as an 8-digit date) and (2) as “latest”. Thus, general users can always get the latest code by pulling `davidturner53/tropoe:latest` (and this would be the `imageName` you use on that command line). However, if you desire to run an older version of the

code for some reason and you have pulled it down to your computer using the date it was built (e.g., davidturner53/tropoe:20220809), then you can execute that version by using that as the `imageName` instead.

## Running the code on Windows machines

This still needs to be developed and tested. We will use an ASCII file to pass in the input arguments. Details are coming...

## Running the code on HPC machines using Singularity

Many high-performance computing (HPC) systems have multiple processing nodes, and it is impractical to run a Docker container on these as that container would need to be installed on each processing node. However, the application Singularity helps this issue, as it embeds the Docker container within a Singularity container where the latter is stored on a local disk (vs in some hidden directory within the compute system). To build a singularity container named “TROPoe.singularity.sif” in the home directory, execute this on the command line:

```
singularity build --fix-perms ~/TROPoe.singularity.sif
docker://davidturner53/tropoe:latest
```

After this new Singularity container is built, then you can execute the code using this command:

```
1 singularity run -B workingDir:/data --env
  yyyyymmdd=ymd,vfile=/data/tropoe/vip.txt,pfile=prior.MIDLAT.nc,shour=00,ehour=24
  ~/TROPoe.singularity.sif
```

which assumes the environment variable `ymd` is the 8-digit date that you want to process, `workingDir` is the location that you want to bind (mount) to `/data`, and that you want to use the mid-latitude prior.

## Controlling the execution of the retrieval

The philosophy behind TROPoe is that most users will configure the retrieval for a given site once, and then process a large amount of data with it. Thus, we wanted to make this processing mode as easy as possible, which is why both the input date and the prior file is passed in with the command line. The other parameters which are used to configure the retrieval are set in the VIP file.

## Command line arguments

The command line arguments are the date, VIP file, prior file, the start / end hours (which are usually 0 and 24, respectively, for most users), verbose (which is usually set to 1 for minimal noise; the other values are used to help debug things), and the name of the container image to run. The prior file is given via the command line as the user may decide to use seasonal a priori information, as in many locations the mean conditions during the winter are markedly different than during the summer.

Note that if the end hour is smaller than or equal to the start hour, the retrieval will perform a single retrieval (i.e., for one observation time) at the start hour. This can be very handy when the user would like to perform a retrieval to compare against another observation (e.g., a collocated radiosonde profile).

## VIP file – high level overview

The Variable Input Parameter (VIP) file sets the configuration of the retrieval. We use a series of key-value pairs within the code, each of which is set to a default value. The VIP file allows the user to overwrite the values of these keys, and thus change the default behavior of the retrieval. There are a certain number of entries that must be in every VIP file; for example, the path to the data used in the retrieval is absolutely required otherwise the code will not be able to find the input data! We have grouped these key-value pairs into two groups: the basic group and the experimental group. For most users, only the basic group are needed. For advanced users, keywords in the experimental group might be changed (although we highly recommend you communicate with the TROPoe authors before you use these experimental keywords).

The VIP keywords are organized in “blocks”. For example, there are 18 VIP entries associated with the IRS (7 of which are considered basic and the other 11 experimental), and 12 VIP entries for a zenith-pointed MWR. Many MWRs also collect data at off-zenith angles (i.e., elevation scans), and these data can be included in the retrieval using another VIP keyword block.

## VIP File (operational processing)

To run the retrieval, relatively few of the key-value pairs need to be set in the VIP file. To see the list of these “standard” VIP entries, execute the code using the number 0 for the date.

For the sake of this discussion, we will assume that the user desires to process observations from an infrared spectrometer through the code. Here are the key-values pairs that need to be set in the VIP file, and what they are doing.

```
tres = 10                                # Temporal resolution [min], 0
    implies maximum (native) temporal resolution
2
station_lat = 36.6                        # Station latitude [degN]; if negative
    get value from IRS/MWR data file
4 station_lon = -97.5                     # Station longitude [degE]; if
    negative get value from IRS/MWR data file
station_alt = 316.0                       # Station altitude [m MSL]; if
    negative get value from IRS/MWR data file
6 station_pres = 980.0                    # Station pressure [mb]; will be only be
    used if there is no other Psfc input

8 irs_type = 1                            # Specifies the type of IRS data to
    read, in this case it is the ARM AERI data
irschl_path = /data/aerich1               # Path to the IRS ch1 radiance files
10 irssum_path = /data/aerisum             # Path to the IRS summary files
irseng_path = /data/aerieng               # Path to the IRS engineering files
12
cbh_type = 1                             # Specifies the type of CBH data to
    read, in this case it is ARM ceilometer
14 cbh_path = /data/ceil                  # Path to the CBH data
cbh_default_ht = 2.0                      # Default CBH height [km AGL], if no
    CBH data found
16
output_rootname = tropoe_retrieval        # String with the rootname of the
```

```

    output file
18 output_path = /data/tropoe                # Path where the output file will be
    placed
20 globatt_Site = Some metadata about the site
    globatt_Dataset_Contact = Some metadata about who created these data

```

As you can see, this configuration will result in retrievals from the ARM AERI using ARM ceilometer data to specify the cloud base height, and the retrievals will be performed at 10-minute resolution. The retrieval always needs to be able to specify a cloud (should the input radiance observations suggest a cloud is needed to get radiative closure), and thus a default cloud base height needs to be specified in case the ceilometer doesn't observe a cloud. [More on this later]. Note anything after the `#` is considered to be a comment

Keywords that are always required by the retrieval are these:

- **tres** – defines the temporal resolution of the retrieval. Use 0 if you desire the maximum temporal resolution of the master dataset
- The three **station\_{lat,lon,alt}** keywords listed above, so that the output file can properly contain this information
- **station\_pres** - defines the default surface pressure for the station, should the atmospheric pressure not be found in the other datastreams. Note that the code will first try to get the pressure from the co-located surface met station (if that keyword block is enabled – more on that later) and the values are valid. It will then try to get the pressure from the master input dataset. Only if these are both missing with this default pressure be used.
- The three **cbh\_** keywords. The retrieval must know where to place the cloud vertically, should the radiance and other observations suggest there is a cloud. More on this keyword block later
- The two **output\_** keywords. The code needs to know how to name the output and where to write it. In this example, the output file will have the name **tropoe\_retrieval.yyyymmdd.hhmmss.nc**.
- There can be any number of **globatt\_XX** key-value pairs as long as the keys are unique. These are additional metadata that are added to the global attributes of the output file. We strongly recommend you use these to provide additional metadata for this dataset you are creating. Note that the entire list of VIP entries will be added to the global attributes of the output file, so that any file created by TROPoe contains the configuration used. The global attributes will also include the version number and package identification number of TROPoe (again, to ensure reproducibility).

TROPoe is designed to have a passive remote sensor (either an IRS or MWR) serve as the “master instrument”, and all other observations (if any) are interpolated to the master’s sampling time. If the temporal resolution (**tres**) is set to zero, then retrievals will be performed for each sample in the master dataset. If the retrieval is set to 10-min resolution, then the code will look for samples in the 00:00-00:10 period, 00:10-00:20, ... 23:50-24:00 and process them. When **tres** > 0, the time associated with the retrieval is the center of the averaging period.

As indicated above, the key-value pairs in the VIP file are organized into blocks. In the above example, there are 6 blocks. We will go into additional details about the various standard keywords that are in each block in the instrument sections below. Experimental keywords, which exist in each block, are described in the appendix.



# Overview of typical input observations

As one of the motivations for TROPoe was to provide an improved retrieval for the ARM AERIs, and due to the fact that D.D. Turner has been part of the DOE ARM program since the mid-1990s, the format of the output file is guided by the ARM data philosophy. Similarly, for each observation type, there will be readers (options) to allow the ARM data to be read in easily. The primary feature of the ARM data format is the time fields: “base\_time” is the number of seconds since 1 Jan 1970 at 00:00:00 UTC, and “time\_offset” is an array of times that are added to base\_time to specify the actual times for each sample in the output file. To make it easier for the user, we only include a maximum of a single day worth of data in a netCDF file, and the 8-digit date is given in the filename. We also include the field “hour” in the output file, which is the hour from 00 UTC on the given date.

The original version of this retrieval was called AERIoe, which had the AERI serving as the master instrument and all other observations were interpolated to the sample times of the AERI. However, we can also run the retrieval in MWR mode (i.e., without an IRS instrument), but this requires an additional key-value pair in the VIP file to indicate that the MWR is the master instrument as it is possible (and often done) to include both the IRS and MWR in a single retrieval. If you desire to use the MWR as the master instrument (i.e., there won’t be an IRS dataset included), then this key-value pair must be added to the VIP: `irs_type = -1`.

## Infrared spectrometer

### Basic instrument / measurements

At the moment, TROPoe is configured to use downwelling spectral infrared radiance data from two instruments: the AERI (manufactured by ABB) and the ASSIST (manufactured by LRtech). (The mention of any particular vendor does not imply endorsement in any way). The ASSIST and AERI are essentially identical instruments from both a philosophical and dataset perspective. Both instruments measure downwelling spectral radiance using an interferometer with a 1 cm maximum optical path delay, regularly view two high-emissivity blackbody targets operating at a warm (usually 60 degC) and ambient temperatures, account for instrument issues like detector nonlinearity, finite field-of-view self-apodization, and spectral calibration using a metrology laser. Due to the use of complex arithmetic in the calibration process (Revercomb et al. 1988), the uncertainty for each observed infrared spectrum is provided and is an important input (along with the radiance spectrum itself) to TROPoe. Explicit details on the hardware and calibration software of the AERI are provided by Knuteson et al. 2004a, 2004b, and Minnett et al. 2000. A paper describing the details of the ASSIST is not yet published, but the calibration approach is very similar to the AERI (Luc Rochette, personal communication). The key point here is that we can model the AERI and ASSIST using an identical approach in the forward model within TROPoe.

As part of the on-board processing by the instrument, the AERI and ASSIST each product three primary files per day. These are:

- The “channel 1/A” radiance file, which is the spectral radiance from the MCT detector and typically spans from 530 to 1800 cm<sup>-1</sup>. For some “extended range systems, the lower wavenumber region may start at 400 cm<sup>-1</sup>. Some important calibration data is stored in this file.
- The “channel 2/B” radiance file, which is the spectral radiance from the InSb detector and typically spans from 1800 to 3000 cm<sup>-1</sup>. Note that the channel 2 data are not currently being used by TROPoe.
- The “summary” file, which contains spectral uncertainty for each radiance observation and a lot of the instrument housekeeping data.

The ARM program decided in the mid-1990s to split the summary file into two data streams, which they call “summary” and “engineering”. Ultimately, TROPoe needs input from both of those data streams. However,

the raw AERI data has all of the needed housekeeping in the one summary file. Some groups, such as the CLAMPS team, process the raw AERI summary file into a single netCDF file, and thus we will list the same path in the VIP file for both. Details on this below.

## The IRS block in the VIP file

Thus, the AERI data may come in slightly different file formats depending on if ARM ingested the data to netCDF or one of the various ingest scripts used by the community did. Furthermore, the ASSIST data format is slightly different still. Thus, the first keyword needed in this block is “irs\_type”, which will specify which type of data that is being read. This keyword currently can assume these values:

- 1 - the data are in the ARM data format, and we will need to read in the irsch1, irssummary, and irsengineer files Note: TROPoe can also operate using irsch2 data (i.e., data from the 1800-3000 cm-1 range that is available from the ASSIST and AERI). Using data from channel 2 requires that different spectral regions are selected using the VIP keyword “spectral\_bands”. However, using channel 2 data is very exploratory at this time and is not encouraged without discussion with the TROPoe authors
- 2 - the raw AERI data were converted to netCDF using the University of Wisconsin – Madison / Space Science Engineering Center (SSEC) “dmv2cdf” application, which is part of their “AERI Armory”. This expects the channel 1 files to be named yyyyymmddC1\_rnc.cdf and the summary files to be named yyyyymmdd\_sum.cdf. The engineering path must point to the summary file
- 3 - the raw AERI data were also converted by dmv2cdf, but the files are named differently: channel 1 files are yyyyymmddC1.RNC.cdf and summary files are named yyyyymmdd.SUM.cdf. Again, the engineering path must point to the summary files
- 4 - the raw AERI data were converted by dmv2cdf, but the channel 1 and summary files were renamed to conform to the ARM naming convention. Again, the engineering path must point to the summary files
- 5 - the native ASSIST netCDF files produced by the instrument
- -1 - this is a special entry that is used to specify that (a) there will be no IRS data input and (b) that the MWR is consider the “master” instrument. It is likely this particular option will be removed and a new more intuitive keyword added in the future.

Assuming the `irs_type > 0`, then the code will read in IRS data. (Note that we anticipate updating the codebase to reading in other IRS dataset, such as the REFIR-PAD, in the future and thus additional `irs_type` values will be added). The next few keywords are required to find the needed data:

- `irsch1_path` – the path to the channel 1 data (e.g., `/data/sgpaerich1C1.b1`)
- `irssum_path` – the path to the summary data
- `irseng_path` – the path to the engineering data, but this should be identical to the `irssum_path` for `irs_type = {2,3,4,5}`.

The AERI and ASSIST both use a gold-plated scene mirror to direct radiance from different directions into the interferometer. There are versions of both instruments that look in multiple elevations. For example, during the RHUBC-2 field campaign (Turner and Mlawer 2010), the AERI was configured to collect data from both zenith and 60 degrees off zenith. The ASSIST is designed to provide not only zenith radiance observations, but also provides radiance observations of both black bodies in its channel 1 output file. Both instruments include a field in the channel 1 file that is named `sceneMirrorAngle` that captures the elevation angle of this mirror when the sample was collected. TROPoe needs to know what angle is associated with the zenith angle. Typically, this is 0 deg for the ASSIST and 180 degrees for the AERI. Thus, we need to specify the value of this keyword in the VIP file to identify the proper zenith angle for TROPoe:

- `irs_zenith_scene_mirror_angle` – the unit of this is degrees.

However, there are times when the AERI's `sceneMirrorAngle` is set to -9999; fortunately, in this case, all of the radiance data are zenith views so setting the value of this keyword to -9999 will allow the code to process the data correctly.

There is a very simple quality control check applied that identify obviously bad spectra before the processing occurs. This test looks at the mean brightness temperature in the 675-680 cm<sup>-1</sup> window, which provides an estimate of the near surface air temperature. There are two keywords that specify the minimum and maximum temperature; observations outside these bounds are assumed to be bad and are skipped. If the environment is significantly warmer or colder than the default values for these two keywords, then add the keywords to the VIP file and redefine their limits to something that is more appropriate for the environment.

```
2 irs_min_675_bt = 263 K  
irs_max_675_bt = 313 K
```

Another keyword that may be useful is `irs_noise_inflation`. As indicated above, both the AERI and ASSIST provide estimates of the random uncertainty in the observation, and this is used for the retrieval. However, if the user expects that the code is trying to overfit the spectral radiance (i.e., that the noise estimate may be too small), then this keyword can be used to increase the observed noise by a multiplicative factor. Generally speaking, we consider this more of an experimental keyword and it isn't used in typical operational processing.

There are many other experimental keywords in the IRS block. These are primarily associated with performing sensitivity tests associated with various calibration factors used in the IRS calibration. These are discussed in the appendix.

## Noise filtering

When the AERI was first developed, the standard sampling strategy was to view the sky continuously for 3 minutes, and then each calibration blackbody for 2 minutes, resulting in a single spectrum every 7 minutes. However, this was found to convolve clear and cloudy radiances into the average spectrum, which makes it much harder for retrieval algorithms to separate out the signals. In the early 2000's, the sampling strategy changed to "rapid scan" mode, wherein the sky averaging period was reduced to ~15 seconds; typically, 6 to 8 sky samples are collected before the blackbodies are viewed. This results in a markedly larger random noise level in the AERI data. The ASSIST has chosen to use a similar rapid scan sampling strategy. To reduce the random error, a principal component noise filter for IRS spectra was developed (Turner et al. 2006). We highly recommend that this noise filter be applied, especially if the data you are analyzing were collected in broken sky conditions. This noise filtering is applied to the channel 1 radiance data before TROPoe is executed. Eventually, this capability will be made available via another container; in the meantime, contact D.D. Turner for implementation details if you desire to use it.

The sampling strategy of the IRS instrument, whether or not it has had the noise filter applied, and desired the temporal resolution of the retrieval all should be considered when setting an important keyword "`avg_instant`", which can take on 3 values:

- 1 – take the instantaneous sample closest to the middle of the averaging period
- 0 – average all of the spectral observations found in the tres averaging period, and decrease the average noise value by the square root of the number of samples in the retrieval
- -1 – average all of the spectral observations found in the tres averaging period, but use the average noise in the retrieval (i.e., do not decrease it by  $\sqrt{N}$ ).

A few thoughts here. If `tres = 0`, then all three of these options for `avg_instant` yield the same results. However, if `tres > 0` then we recommend you either use `avg_instant = 1` or `avg_instant = -1` in your

processing, as we have found that using `avg_instant = 0` results in TROPoe overfitting the data and the quality of the resulting retrievals is poor. Using `avg_instant = 1` is recommended if the IRS data have been noise filtered, as using `avg_instant = -1` will convolve clear and cloudy scenes in the averaging process (if the sampling period has broken clouds overhead).

## Microwave radiometer

### Basic instrument / measurements

There are a multitude of ground-based microwave radiometers available to the operational and research community. For thermodynamic profiling, the most common models are the MP-3000 (manufactured by Radiometrics; Solheim et al. 1998) and the HATPRO (manufactured by Radiometer Physics; Rose et al. 2005). (Again, mention of any particular vendor does not imply endorsement in any way). Both of these radiometers make multiple measurements along the side of the 22.2 GHz water vapor absorption line (i.e., the K-band) and the side of the 60 GHz oxygen complex (i.e., the V-band). However, radiometers have also been built to take advantage of the much stronger 183.3 GHz water vapor band, especially for profiling in the Arctic (e.g., Cimini et al. 2009). Additionally, some MWRs have only a few channels (e.g., ARM’s two and three channel systems; Cadeddu et al. 2013), and including observations from any of these MWRs provides additional constraints to the retrieved profiles and cloud properties. Thus, TROPoe was designed to use any set of microwave frequencies.

In all of these cases, the basic measurement made by the MWR is the downwelling sky brightness temperature at several frequencies. Often, the MWR is collecting data at multiple elevation angles, as this increases the optical depth along the path and thus can provide additional information to the retrieval (e.g., Crewell and Löhnert 2004). TROPoe uses two different VIP blocks to handle the zenith and off-zenith (i.e., elevation scan) observations.

The sky brightness temperature observations (henceforth called `Tbsky`) can be arranged in the input file a couple different ways. One way is to store the data as a set of 1-dimensional time-series, with each frequency having a different field name; e.g., the ARM 2-channel microwave radiometer data files have the fields `tbsky23` and `tbsky31`. A second way is to have a single 2-dimensional field, perhaps called `tbsky`, where the two dimensions are time and frequency. In this case, we need to provide in the VIP file the name of the field that contains the frequency information. In both of the previous cases, elevation scans may be interspersed within the zenith observations and thus a separate field is used to identify the elevation angle for each sample (the name of this field must also be specified in the VIP file). A third case, which is used heavily in Europe, is where the MWR data have been quality controlled using software developed at the University of Cologne; a trademark of this format is that they use a 3-dimensional matrix to encode the elevation scans as a function of time, frequency, and elevation.

The MWR community has typically used an elevation angle of 90 degrees as the zenith angle. We adopt that strategy here, and only data that point in this direction will be included using the configuration in this `mrw_type` block.

Unlike the IRS observations, most MWRs do not provide uncertainty estimates for the observed brightness temperatures. Thus, the user needs to indicate, via the VIP, the uncertainty level that should be assumed for each frequency. TROPoe will assume that these uncertainties are uncorrelated.

The bandpass typically used in MWRs is between 50 to 1,000 MHz, depending on vendor and frequency channel. TROPoe models each channel using a central wavelength; i.e., it does not perform a high-spectral resolution run which is then convolved with the filter bandpass. As some radiometers use a double-side passband (i.e., measuring two specific bandpasses on either side of a set frequency point), this does possess challenges in properly simulating the radiometer’s observation. TROPoe has adopted the perspective that

each MWR frequency channel can be represented with a single “center wavelength”, wherein that frequency is specified in the VIP file for each brightness temperature frequency.

Lastly, many MWRs have exhibited what appears to be radiometric biases in their brightness temperature observations. TROPoe allows the user to specify, again via the VIP, a brightness temperature offset that should be added to the observation at each frequency before the retrieval begins.

## The MWR block in the VIP file

- `mwr_type` – the type of zenith pointing data
  - 1 – all Tbsky data arranged as a series of 1-dimensional time-series.
  - 2 – all Tbsky data arranged as a single 2-dimensional field (frequency by time)
  - 3 – the data arranged following the Univ Cologne data format
- `mwr_path` – the path to the MWR data (again, starting from `/data`)
- `mwr_rootname` – the rootname of the MWR data file
- `mwr_elev_field` – the name of the elevation field. If all data are zenith, then omit this
- `mwr_freq_field` – the name of the frequency field, if `mwr_type` = 2
- `mwr_n_tb_fields` – the number of Tbsky fields to use in the retrieval
- `mwr_tb_field_names` – the name of the Tbsky field(s). If `mwr_type` = 2, then this is a single name. If `mwr_type` = 1, then a comma-delimited list with the name of the `mwr_n_tb_fields` in the netCDF file to read (note that the order of the names should match the order of the frequencies listed below)
- `mwr_tb_field1_tbmax` – a very simple QC test, wherein the observed brightness temperature of the first field listed above must be below this value. [D.D. Turner uses a simple rule-of-thumb wherein if the observed Tbsky at 31 GHz is above 100 K, then it is raining; thus, he usually lists this frequency first and sets this threshold to 100 K]
- `mwr_tb_freqs` – a command-delimited list of central frequencies, in GHz, to use for each of the `mwr_n_tb_fields` channels
- `mwr_tb_noise` – the random noise level, in K, to assume for each of the `mwr_n_tb_fields` channels
- `mwr_tb_bias` – the bias error, in K, to assume for each of the `mwr_n_tb_fields` channels. This will be added to the observed Tbsky values

## MWRscan\_type (describe each, including format)

If the MWR is collecting data at elevation angles other than zenith, then the user can include these observations using this block. As indicated above, the MWR community has traditionally used 90 degrees as the zenith elevation. Some MWRs only scan in a single half-plane (i.e., from near 0 degrees to 90 degrees), whereas others scan in the entire plane (i.e., from 0 degrees through 90 degrees to near 180 degrees). Typically, these full-plane scans are done with “equal elevation” angles on both sides of zenith; i.e., if there is a 10-degree observation there will also be one at 170 degrees. TROPoe assumes the atmosphere is plane-parallel when treating these elevation scans, and thus these equal elevation scans are modeled exactly the same way. Azimuthal scans are ignored, for the same plane-parallel assumption.

The MWRscan block has a very similar set of keywords as the `mwr_type` block above; just use “`mwrscan_`” in place for each “`mwr_`” in the above list. Note that the number of Tbsky fields used in the `mwrscan_` block does not have to be the same as in the `mwr_` block; for example, Crewell and Lohnert (2004) recommend only using the more-opaque channels in the V-band in elevation scans. However, we do recommend that if you use some of the same channels in both the `mwr_` and `mwrscan_` blocks that they have the same center frequencies, noise levels, and Tb bias offsets. There are two new keywords that are unique to the `mwrscan_` block that are not in the `mwr_` block. These are associated with the elevation scans:

- `mwrscan_n_elevations` – the number of elevation angles to use

- `mwrscan_elevations` – the elevation angles to use in the retrievals (comma-delimited)

As an illustration, suppose that a MWR is regularly scanning and the elevation scans are 10, 25, 40, 90, 140, 155, and 170 (which are then repeated ad nauseum). The 90-degree scan should be handled using the `mwr_` block. Suppose you would like to include only the 10, 25, 40, and 140 degree scans in your retrieval (e.g., let's pretend that the 155 and 170 degree data are obscured by trees or something). Then the `mwrscan_n_elevations` should be equal to 4, and the `mwrscan_elevations` = 10, 25, 40, 140.

## Bias determination / correction

Most MWRs use an internal noise diode to provide a second reference point (the primary reference being an internal ambient temperature blackbody target) for calibration. There are multiple ways to calibrate the noise diode; e.g., so-called TIP curves and external liquid nitrogen (LN2) cooled blackbodies. Both methods have strengths and weaknesses. However, many radiometers demonstrate drift in the radiometric calibration of one or more channels with time. This can result in spectral Tbsky observations that are aphysical; i.e., there does not exist an atmospheric state that will satisfy the spectral observations. This is one example where a bias correction scheme can be useful. The second is when the sky is clear but the retrieval is providing LWP » 0 g/m<sup>2</sup>; this is usually attributed to a bias in the more transparent K-band channels. The `mwr_tb_bias` and `mwrscan_tb_bias` keywords enable the user to apply a brightness temperature offset to one or more channels before the retrieval is performed. Details on how to determine these biases is shown in the appendix.

## Surface met

Many sites where either IRS or MWRs are located has surface meteorological stations that measure atmospheric pressure, temperature, and relative humidity with in-situ probes. Indeed, most MWRs have sensors like this integrated into the instrument. These met observations provide additional information to the retrieval, and are especially useful for MWR-only retrievals (e.g., Guy et al. 2022). Here are the keywords needed to include surface met observations into the retrieval:

- `ext_sfc_temp_type` – one of multiple values, indicating what data format to expect
  - 1 – use ARM met data
  - 2 – use NCAR ISFS data
  - 3 – use CLAMPS MWR met data
- `ext_sfc_temp_rep_error` – size of the representativeness error (deg C) to add to the surface met observation
- `ext_sfc_wv_type` – one of multiple values, indicating what data format to expect
  - 1 – use ARM met data
  - 2 – use NCAR ISFS data
  - 3 – use CLAMPS MWR met data
- `ext_sfc_wv_mult_error` – multiplier to scale the random error of the surface water vapor mixing ratio uncertainty
- `ext_sfc_wv_rep_error` – size of the representativeness error (g/kg) to add to the surface water vapor mixing ratio observation
- `ext_sfc_path` – path to the surface met data file (e.g., /data/surface\_met). Note this path is used for both the temperature and water vapor data
- `ext_sfc_relative_height` – height of the surface station relative to the height of the IRS scene mirror (if `irs_type` > 0) or MWR (if `irs_type` < 0)
- `ext_sfc_pres_type` – one of multiple values
  - 0 – use the internal IRS pressure (if `irs_type` > 0) or MWR pressure
  - 1 – use ARM met data

- 2 – use NCAR ISFS data
- 3 – use CLAMPS MWR met data

The code is currently set to read three basic file types. Here are the fields needed from each:

1. ARM met: base\_time, time\_offset, atmos\_pressure [kPa], temp\_mean [degC], and rh\_mean [%RH]
2. NCAR ISFS: base\_time, time\_offset, pres [hPa], tdry [degC], rh [%RH]
3. CLAMPS MWR met data: base\_time, time\_offset, sfc\_pres [hPa], sfc\_temp [degC], sfc\_rh [%RH]

Note that, for the surface met, the basic assumption is that the temperature is measured with a random uncertainty of 0.5 degC,

## Basic instrument / measurements

In-situ meteorological measurements are very common, and typically measure atmospheric pressure, ambient temperature, and relative humidity. (Wind and precipitation observations, which are also made by most met stations, are not used by TROPoe).

### ext\_sfc\_XX\_type (describe each)

TROPoe has the ability to read in in-situ met data from a range of different sources. These are the sources that are currently available within TROPoe, and the fields that are read in:

- ARM surface met data: base\_time (single value, time since 1970-01-01 00:00:00 UTC), time\_offset (array, seconds since base\_time), atmos\_pressure (array, kPa), temp\_mean (array, degC), and rh\_mean (array, %RH).
- NCAR ISFS met data: base\_time (single value, time since 1970-01-01 00:00:00 UTC), time (array, seconds since base\_time), pres (array, hPa), tdry (array, degC), and rh (array, %RH)
- MWR met data: base\_time (single value, time since 1970-01-01 00:00:00 UTC), time\_offset (array, seconds since base\_time), {sfc\_pres,p\_sfc} (array, hPa), {sfc\_temp,t\_sfc} (array, degC or degK), and {sfc\_rh,rh\_sfc} (array, %RH). Note this is the only option for the surface met input where the code checks the units of the temperature field to look for “K”, “k”, or “Kelvin” and if found, will convert it to Celcius. Note that the moisture field is always assumed to be in relative humidity [units %RH].

## Assumed uncertainties

The user has the ability, via the VIP file, to specify the random error in the in-situ temperature and relative humidity observations. This is done via the VIP keywords ext\_sfc\_temp\_random\_error and ext\_sfc\_rh\_random\_error. The default values for these two VIP values are 0.5 degC and 3.0 %RH, respectively. The uncertainties are propagated in the standard way to provide the uncertainty in the surface-level water vapor mixing ratio value. Note that the ambient pressure is assumed to have zero uncertainty.

## Ceilometer

The retrieval will always assume there is a cloud somewhere in the atmosphere (unless the user disables the cloud retrieval by setting both retrieve\_lcloud and retrieve\_icloud to zero in the VIP file – which is not recommended). Thus, the algorithm needs to have some information on how high to place this cloud, should it exist. (It will not exist if the retrieved liquid water path and ice cloud optical depth are zero within their

uncertainties). Note that currently the measured cloud base height from the ceilometer (or the default value, if that is used) is assumed to have no uncertainty.

## Basic instrument / measurement

The ceilometer is a simple attenuated backscatter lidar that is designed to identify the lowest height that has a markedly larger backscatter return above the background value. Ceilometers are commercially available from many different vendors, and are typically deployed around airports. Many experimental research sites will include ceilometers as part of the instrument complement due to their rugged automated capability, simplicity of use, and the importance of having observations of the cloud base height. The desired unit for cloud base height within TROPoe is km above ground level (AGL), where the 0 km AGL is the height of the master instrument (either the IRS or MWR).

## Possible formats

There are multiple different readers implemented within TROPoe, selected using `VIP.ceil_type`:

1. ARM ceilometer data: `base_time` (single value, time since 1970-01-01 00:00:00 UTC), `time_offset` (array, seconds since `base_time`), and `first_cbh` (in m AGL). Files should be named *ceilyyyymmdd\**.
2. ASOS/AWOS ceilometer data (also known as the Blumberg ingest): `base_time` (single value, time since 1970-01-01 00:00:00 UTC), `time_offset` (array, seconds since `base_time`), and `cloudBaseHeight` (in km AGL). Files should be named either *ceilyyyymmdd\** or *cbhyyyymmdd\**.
3. NOAA CLAMPS Doppler lidar fixed point (zenith) data: `base_time` (single value, time since 1970-01-01 00:00:00 UTC), `time_offset` (array, seconds since `base_time`), and `cbh` (in km AGL). Files should be named either *dlfpyyyymmdd\**.
4. ARM Doppler lidar vertical motion moment datastream: `base_time` (single value, time since 1970-01-01 00:00:00 UTC), `time_offset` (array, seconds since `base_time`), and `dl_cbh` (in m AGL). Files should be named *dlprofstatsyyyymmdd\**.
5. JOYCE Vaisala ceilometer data: `base_time` (single value, time since 1970-01-01 00:00:00 UTC), `time_offset` (array, seconds since `base_time`), and `first_cbh` (in m AGL). Files should be named *ct25kyyyymmdd\**.
6. JOYCE Jenoptic ceilometer data: `time` (array, unix time + 2.0828448e+09 in seconds), `cbh` (in m AGL). Files should be named *CHMyyyymmdd 7.E-PROFILE ceilometer data: time (array, unix time in days), cloud\_base\_height (a 2-d matrix, will use the first level, units of m AGL, and vertical\_visibility (array, units m AGL). Files should be named L2yyyyymmdd\**. If the CBH  $\leq 0$  and visibility  $> 0$ , then the latter will replace the former.

## Importance of the prior information

### Thermodynamic prior covariance matrix (external to code)

The retrieval of thermodynamic profiles from passive spectral radiance data is a mathematically ill-posed problem; namely, there are many different solutions (i.e., profiles) that if input into a radiative transfer model would agree with the observation within its uncertainty. However, many of these solutions are not physically realistic. For example: suppose the true temperature profile follows a standard lapse rate; a profile that oscillates back and forth with height around this true profile could be possible solution mathematically, but that type of profile isn't observed in nature. Stated a different way, there is correlation between any two levels in the atmosphere, and incorporating information about this correlation helps to constrain the derived solution to a more physically realistic profile. This is the job of the prior covariance matrix.



The prior covariance matrix is computed from a historical collection of radiosonde profiles. Each of these profiles is first interpolated to a fixed vertical grid, and then the covariance matrix is computed. The covariance matrix includes the covariance between temperature (the matrix TT) between any two levels  $i$  and  $j$ , the covariance of water vapor (the matrix QQ) between any two levels  $i$  and  $j$ , and the covariance between temperature at level  $i$  and water vapor at level  $j$  (for any levels  $i$  and  $j$ , the matrix TQ). The matrix should be visualized like this:

```
S_a = \begin{bmatrix} TT & TQ \\ QT & QQ \end{bmatrix}
```

where  $QT = \text{transpose}(TQ)$ .

TROPoe has three prior covariance matrices included within the package: `prior.TROPICS`, `prior.MIDLAT`, and `prior.POLAR`. The user has the option to use their own prior covariance matrix also.

The three embedded prior covariance matrices are computed on a vertical grid that has finer resolution at the surface that coarsens with height; this is to capture finer detail near the surface where there is higher information content in the IRS and MWR observations. If the user wants to use their own prior dataset, we encourage them to use a similar geometrically expanding-with-height vertical grid.

## Vertical grid

TROPoe allows the user to specify the vertical grid that they desire to use in the retrieval. This is done via the VIP keyword `zgrid`. This comma delimited array, which has units km AGL, must have its first value at 0, its last value below the maximum height in the prior covariance file (which is 20 km for the three standard atmospheres), and be monotonically ascending. TROPoe will interpolate the prior covariance matrix to this input `zgrid`.

## Prior recentering

The prior dataset consists of a mean profile of temperature and water vapor mixing ratio, and a matrix that describes the covariability around this mean. However, if the current conditions are markedly different than the prior mean profile, then the biases in the retrieved profile (especially in regions where the signal in the observations is low) can result. To help mitigate this, we borrow a technique from data assimilation used within numerical weather prediction centers. In particular, TROPoe has the ability to recenter the prior data. This is done by adjusting the mean profile, and then modifying the covariance matrix accordingly. The goal is to reduce the bias in the mean profile enough that the retrieved profile is not biased by the difference between the mean prior and the true conditions.

The VIP keyword `recenter_prior` is used to turn on / off the recentering. Its default is on.

Of the two variables, the signal-to-noise for water vapor is weaker than for temperature in both the IRS and MWR. Thus, our approach is to recenter the mean water vapor first. This is done by using the input surface met data and comparing the daily mean in-situ value with the lowest level in the prior. The ratio of these two values, call this SF, is used as a height-independent multiplier to the entire prior mean profile, which brings the adjusted mean water vapor profile in closer agreement with the current conditions. Note that the magnitude of the QQ, TQ, and QT portions of the covariance matrix are scaled by SF<sup>2</sup>, SF, and SF, respectively, thereby maintaining the relative magnitude of the covariance around the mean. The new mean temperature profile is computed as the temperature required to yield the same relative humidity profile as was in the original covariance matrix. Since the relative change in the temperature is small (on the Kelvin scale), no adjustment is applied to the TT, TQ, or QT parts of the covariance matrix.

Suppose that the user does not have any independent surface met data, yet would still like to recenter the prior. There are two options. The default fall-back option is to first estimate the near-surface temperature

value from the brightness temperature derived from the IRS (using the 675-680 cm<sup>-1</sup> region), or if the IRS is not available from the highest frequency channel in the 56-60 GHz region from the MWR. The daily mean of this near-surface temperature estimate is computed. Again, assuming that the RH profile from the original prior is to be conserved, a new surface water vapor mixing ratio value is computed that would be in agreement with this derived radiometric near-surface temperature value. This new water vapor value is used to compute SF, and the logic from the above paragraph is used.

The user has the ability to force TROPoe to use any desired value for the input water vapor mixing ratio, and thus control the SF value. This is done via the VIP keyword `recenter_input`.

## Specifying the prior

The prior is specified by the user on the command line when executing the retrieval. If the user specifies “prior.TROPICS”, “prior.MIDLAT”, or “prior.POLAR” then the corresponding embedded prior will be used. If the command line option is some other value, the retrieval assumes that the user input the path/filename of the prior that should be used.

## Cloud prior covariance information

The prior covariance discussed above was in the context of temperature and water vapor. However, TROPoe is also able to retrieve cloud properties (and indeed, we encourage users to always have at least one of `retrieve_lcloud` or `retrieve_icloud`, or both, turned on). This means we need to specify the prior conditions (mean and standard deviation) for the cloud properties that will be retrieved. These values are set in the VIP file. The VIP keywords, and their default values [units], are

```
prior_lwp_mean = 0           # mean liquid water path [g/m^2]
2 prior_lwp_sdev = 200        # standard deviation of liquid water path
  [g/m^2]
prior_lreff_mean = 8         # mean liquid droplets effective radius
  [$/mu$m]
4 prior_lreff_sdev = 4        # standard deviation of liquid droplets
  effective radius [$/mu$m]
prior_itaum_mean = 0         # mean ice optical depth [unitless]
6 prior_itaum_sdev = 50       # standard deviation of ice optical depth
  [unitless]
prior_ireff_mean = 25        # mean ice crystal effective radius
  [$/mu$m]
8 prior_ireff_sdev = 8        # standard deviation of ice crystal
  effective radius [$/mu$m]
```

Note that the signal-to-noise for cloud properties is extremely large in the IRS, and pretty strong in the MWR, so having mean values for `prior_lwp_mean` and `prior_itaum_mean` of zero is not very restrictive (given the very large uncertainty for those two parameters as specified by the `sdev` values). Naturally, the user can override any of these with their own values. These are added to the prior covariance matrix like this:

```
S_a = \begin{bmatrix} TT & TQ & TC \\ QT & QQ & QC \\ CT & CQ & CC \end{bmatrix}
```

where C denotes the cloud properties arranged as [lwp, lreff, itau, ireff]. Note that we assume there is no correlation between T and C or Q and C, and thus  $TC = QC = 0$ .

## Trace gas prior covariance

TROPoe has some limited ability to retrieve trace gases; namely, carbon dioxide (CO<sub>2</sub>), methane (CH<sub>4</sub>), and nitrous oxide (N<sub>2</sub>O). This capability is still very experimental, and thus we recommend that these retrievals be turned off (using VIP keywords `retrieve_co2`, `retrieve_ch4`, and `retrieve_n2o`, respectively) if the objective is to get thermodynamic profiles and cloud properties. The default is for these components to be turned off; to turn one or more of these on, then the retrieve flag should be set to 2 (not 1).

We have very few vertical profiles of these trace gases from which to build a statistically well-defined level-to-level covariance matrix. Thus, if a trace gas (TG) retrieval is enabled, we assume that the shape of the TG profile is a constant in the free troposphere, and a different constant in the PBL (i.e., a “chair” profile). The code actually retrieves a three-element vector for each trace gas: [free troposphere concentration, difference in PBL vs free troposphere concentration, and “shape”]. (Note: the “shape” parameter goes with `retrieve_xxx = 1`; this option was experimental and is no longer supported).

Like the cloud prior information, trace gas prior information needs to be appended to the covariance matrix. These are specified via the VIP file with these keywords, each of which is expecting a 3-element vector:

```
prior_co2_mean = [-1,5,arbitrary]          # units ppm, ppm, and
  unused
2 prior_co2_sdev = [2,15,arbitrary]         # units ppm, ppm, and
  unused
prior_ch4_mean = [1.793,0,arbitrary]       # units ppm, ppm, and
  unused
4 prior_co2_sdev = [0.0538,0.0015,arbitrary] # units ppm, ppm, and
  unused
prior_co2_mean = [0.310,0,arbitrary]       # units ppm, ppm, and
  unused
6 prior_co2_sdev = [0.0093,0,arbitrary]     # units ppm, ppm, and
  unused
```

A couple of details about these default values: \* If a standard deviation is set to zero (whether in the trace gas, cloud, or any section), the assumed intent is that the user does not want the retrieval to actually retrieve that value. So, in this example here, the zero in the second element of the `prior_n2o_sdev` implies that the retrieval should not change the relative PBL concentration at all; i.e., that the N<sub>2</sub>O profile should be treated as a constant with height profile by the retrieval. Note that TROPoe actually will replace this zero with an extremely small number in the processing, as a diagonal matrix (like this part of the a priori covariance matrix) cannot be inverted with zeros on the diagonal. \* The default mean and standard deviation values for free-tropospheric elements of the CH<sub>4</sub> and N<sub>2</sub>O are derived from a multi-year analysis of TCCON data at the ARM SGP site. \* The -1 in the first element of the CO<sub>2</sub> mean value has special meaning. CO<sub>2</sub> has a strong seasonal cycle, and the mean annual value has been increasing steadily for the last 6 decades. TROPoe has a very simple parameterization, built using data from the NOAA site at Mauna Loa, that uses the year and month of the day being processed to estimate the CO<sub>2</sub> concentration in the free troposphere. When this flag is set to -1, this parameterization is used to provide the estimated free-tropospheric value for the retrieval.

The depth of the PBL (PBLH), which separates the PBL from the free troposphere, is derived from the retrieved temperature profile using a lifted parcel approach and the potential temperature profile. There is a minimum (floor) value set for the PBLH; this is set via the VIP keyword `min_PBL_height` [km AGL], which has a default value of 0.3 km. The user can also set a maximum value (ceiling) for the PBLH also using the VIP keyword `max_PBL_height` [km AGL]. # Interpreting TROPoe Output ## Quality control flags, RMSr / RMSa, convergence\_flag, gamma, n\_iters

There are several fields that provide information on the quality of the TROPoe in the output file. The most basic one is the **qc\_flag**. This field has four possible values: \* **qc\_flag** – quality control flag \* 0 – implies the quality is okay \* 1 – implies the hatch was not open for full observation period \* 2 – implies retrieval did not converge \* 3 – implies retrieval converged, but RMS between observed and computed spectrum is too large

If the **qc\_flag** is 1 the retrieval should not be used as the IRS hatch was closed during the observation period and the radiance observed are either from inside the hatch or contaminated from observations of the hatch (in the case of `avg_instant = 1`). A **qc\_flag** of 2 suggests that the user should check the **converged\_flag** variable (which will be described below) for more information about the lack of convergence. This does not necessarily mean that the retrieval is “bad”, but the users will need to do more investigations of the variables that describe the quality of the retrievals to ensure that the retrieval is acceptable. Finally, a **qc\_flag** of 3 indicates that the RMS difference of the radiance observations from the forward calculation of the state vector is larger than the VIP entry `qc_rms_value`. These retrievals are most likely bad and should not be used.

The output files contain two RMS values that describe how consistent the retrieval is with the observations. The first RMS, **rmsa**, is calculated for the entire observation vector

$$\text{RMS}_a = \sqrt{\frac{\sum \left( \frac{Y - F(X_n)}{\sigma_Y} \right)^2}{N_Y}}$$

Notice that the difference of the observation vector from the forward calculation is normalized by the uncertainty of the observation in this calculation to account for the varying levels of uncertainty in the observation vector. **RMSa** is the RMS value that is used to determine the **qc\_flag** and is also monitored during each iteration to check if the retrieval is trending away from the observations. The second RMS value in the output file is **rmsr**, which is the RMS for only the radiance observations in the observation vector (IRS and MWR observations). This is calculated the same way as **rmsa**. The inclusion of **rmsr** in the output allows the user to differentiate whether a high **rmsa** value is caused by the radiance observations or other observation types in the observation vector.

TROPoe checks for convergence in the retrieval using the technique described in Rodgers (2000). The retrieval is determined to converge when the difference in the forward calculation between subsequent iterations scaled by the observations uncertainty is less than 30% of the length of the observation vector. If the retrieval does not converge before the maximum number of iterations is reached (which is set by the VIP entry `max_iterations`), then iteration with the best **rmsa** value is used as the final retrieval. If at any time while the code is iterating the **rmsa** drastically increases over the iteration with best **rmsa**, then the code will stop iterating as the retrieval is likely trending away from the proper solution and the iteration with the best **rmsa** will be used as the final retrieval. Although unlikely to occur, the code will also stop iterating if a NaN is found in the updated state vector. In this scenario, the previous iteration is used as the final retrieval. These different states of convergence (or lack thereof) are stored in the output variable **converged\_flag**.

- **converged\_flag** - Convergence flag
  - 1 – indicates convergence in Rodgers sense (i.e.,  $\text{dim} X \ll \text{dim} Y$ )
  - 2 – indicates best rms after rms increased drastically
  - 3 – indicates best rms after maximum iterations
  - 9 – indicates NaN in updated state vector

In order to account for the first guess state vector potentially being far away from the final solution, a scalar parameter,  $\gamma$ , is applied to increase the weight of the prior in the retrieval during the early iterations. This helps to keep the retrieval stable while the observations with the most information content slowly modify the retrieval towards the correct solution. The value of  $\gamma$  decreases with each iteration until it becomes 1. The rate of decrease in  $\gamma$  is different depends on whether the master instrument is an IRS or MWR and are as follows:

$$\text{IRS } \gamma = [1000, 300, 100, 30, 10, 3, 1, \dots]$$

```
1 MWR \gamma = [100, 30, 10, 3, 1, ...]
```

The code will not check for convergence until  $\gamma = 1$ . However, the iterations can still be stopped if the *rmsa*, drastically increases. Therefore, the value of  $\gamma$  for the final retrieval is stored in the output variable *gamma*.

Finally, with the different ways for the retrieval to finish, it is important to know what iteration the final retrieval is from. This information is stored in the *n\_iter* variable. For a retrieval that converged in the traditional way this will be the last iteration, but when the retrieval does not converge this will be the iteration with the lowest *rmsa*.

## Actual retrieved fields

### Temperature and humidity

Temperature and humidity are two of the retrieved fields that most users are probably interested in. The temperature field is output in Celcius and stored in the output variable *temperature*. The humidity variable for the retrieval is water vapor mixing ratio and is output with units of g/kg and stored in the *waterVapor* output variable. Both temperature and waterVapor are two dimensional fields with dimensions of (time, height).

It is also possible to run TROPoe without retrieving temperature or humidity. In these cases, the VIP entries *retrieve\_temp* and *retrieve\_wvmr* are set to 0. When these fields are not retrieved, the output variables will be equal to the first guess profiles for all retrieval times.

### Cloud properties

TROPoe also has the ability to retrieve cloud properties. The retrieved cloud property variables are:

- *lwp* – Liquid water path in g/m2
- *lReff* – Liquid water effective radius in microns
- *iTau* – Ice cloud optical depth (geometric limit)
- *iReff* – Ice effective radius in microns

All of the cloud properties retrieved fields have a single dimension which is time. Like temperature and humidity, the retrieval can be run without retrieving these cloud properties. This is controlled by the VIP entries *retrieve\_lcloud* and *retrieve\_icloud*. When these fields are not retrieved the output variables will contain the prior mean value for that field. Note: the default VIP entries are *retrieve\_lcloud*=1 and *retrieve\_icloud*=0.

## Uncertainties in the actual retrieved fields

One of the biggest strengths of TROPoe is that the retrieval propagates the uncertainties from the prior and the observations and the sensitivity to the forward model through the calculation of the posterior covariance matrix. The square-root of the diagonal of the posterior covariance matrix provides the 1-sigma uncertainties for the final state vector. These 1-sigma uncertainties for the retrieved fields in the following output file \* *sigma\_temperature* – 1-sigma uncertainty in temperature \* *sigma\_waterVapor* – 1-sigma uncertainty in water vapor mixing ratio \* *sigma\_lwp* – 1-sigma uncertainty in liquid water path \* *sigma\_lReff* – 1-sigma uncertainty in liquid water effective radius \* *sigma iTau* – 1-sigma uncertainty in ice cloud optical depth (geometric limit) \* *sigma iReff* – 1-sigma uncertainty in ice cloud effective radius

The off-diagonal elements of the posterior covariance matrix describe the covariance between different elements of the state vector. These elements show things like how the levels in temperature profile covary, how temperature and water vapor covary, and also how the temperature and humidity profiles covary. The posterior covariance is included in the output file as long as the VIP key `output_file_keep_small=0` (the default value). The posterior covariance matrix's variable name in the output file is ***Sop*** and it has three-dimensions which are (time, arb, arb) where arb is the length of the state vector. There is also an output variable named ***arb*** that describes what retrieved field each element of the state vector and posterior covariance matrix corresponds to.

- ***arb*** – Arbitrary dimension
  - 1 – temperature
  - 2 – water vapor
  - 3 – liquid water path
  - 4 – liquid water effective radius
  - 5 – ice cloud optical depth
  - 6 – ice cloud effective radius
  - 7 – carbon dioxide
  - 8 – methane
  - 9 – nitrous oxide

## Information content in actual retrieved fields

NEED TO DO THIS SECTION

## Derived fields

After the final state vector has been retrieved, the temperature and humidity profiles are used to obtain derived profiles and indices which are included in the output files.

### Profiles

The derived profiles are: \* ***theta*** – Potential temperature (K) \* ***thetae*** – Equivalent potential temperature (K) \* ***rh*** – Relative humidity (%) \* ***dewpt*** – Dew point temperature (C)

These variables all have dimensions of (time, height). When calculating the profiles, the pressure that is used is obtained from the surface pressure data and the hypsometric equation is used to calculate the change in pressure with height

### Indices

The derived indices are all stored in separate variables in the output files. Each variable has dimensions of (time) and there are 10 derived indices that correspond to the following variables: ***pwv*** – Precipitable water vapor (cm) ***pblh*** – Planetary boundary layer height (km AGL) ***sbih*** – Stable boundary layer inversion height in (km AGL) ***sbim*** – Stable boundary layer inversion magnitude (C) ***sbLCL*** – Surface-based lifted condensation level (km AGL) ***sbCAPE*** – Surface-based CAPE (J/kg) ***sbCIN*** – Surface-based CIN (J/kg) ***mlLCL*** – Mixed-layer lifted condensation level (km AGL) ***mlCAPE*** – Mixed-layer CAPE (J/kg) ***mlCIN*** – Mixed-layer CIN (J/kg)

The output file also provides 1-sigma uncertainties for these derived indices in the corresponding ***sigma\_xxx*** variable that has the same dimensions as derived indices. These uncertainties are determined through Monte

Carlo sampling of the posterior covariance matrix to obtain perturbed thermodynamic profiles from which the samples of the derived indices are calculated to obtain the spread of each index.

## Understanding **obs\_flag**, **obs\_dim**, **obs\_vector**, and **forward\_calc**

The TROPoe output files include information on the observations that are used for each retrieval. All of the observations are stored in the **obs\_vector** variable which has dimension of (time, Ydim) where Ydim is the length of the observation vector. The units for **obs\_vector** are mixed as it can contain many different types of observations (e.g. IRS radiances, MWR brightness temperatures, surface met station, etc.). The uncertainties of the elements in **obs\_vector** are stored in **obs\_vector\_uncertainties**. The **obs\_flag** variable indicates the type of observation for each element in **obs\_vector**. There is also an **obs\_dim** variable that provides the wavenumber, frequency, height, etc. of the elements in the **obs\_vector**. The possible values for **obs\_flag** and the corresponding meaning of the values in **obs\_dim** are:

- 1 – Radiance in mw/(m<sup>2</sup> sr cm<sup>-1</sup>) from IRS, **obs\_dim**: wavenumber (cm)
- 2 – Brightness temperature in Kelvin from zenith-pointing MWR, **obs\_dim**: frequency (GHz)
- 3 – Temperature from an external profile (units depend on type), **obs\_dim**: height (km)
- 4 – Water vapor from external profile (units depend on type), **obs\_dim**: height (km)
- 5 – Surface met station temperature (units depend on type), **obs\_dim**: height (km)
- 6 – Surface met station water vapor (units depend on type), **obs\_dim**: height (km)
- 7 – Temperature from a model profile (units depend on type), **obs\_dim**: height (km)
- 8 – Water vapor from a model profile (units depend on type), **obs\_dim**: height (km)
- 9 – Carbon dioxide in-situ obs (units depend on type), **obs\_dim**: height (km)
- 10 – Brightness temperature in Kelvin from scanning MWR, **obs\_dim**: frequency\*100 + elevation/1000
- 11 – Virtual temperature from RASS (units depend on type), **obs\_dim**: height (km)

Also included in the output file is the forward calculation for the final state vector in the variable **forward\_calc**. The forward calculation is the expected value of the observation based on the retrieved state vector. The forward calculation is performed using the forward models discussed in Section 8. The values in **obs\_flag** and **obs\_dim** also apply to **forward\_calc**. Having both the **obs\_vector** and **forward\_calc** in the output allows the user to examine how well the observations are being fitted by the retrieval. For a “good” retrieval the difference between the elements in the **obs\_vector** and **forward\_calc** should be within the **obs\_vector\_uncertainties**. This type of analysis of the observation vector and forward calculation can help the user to pick out which observation might be causing problems in the retrieval.

## Understanding the global attributes

The global attributes of the output files contain the version of TROPoe used for the retrieval, date the retrieval was performed, and references for TROPoe, LBLRTM, and MonoRTM. The global attributes also have information on the prior that was used. Lastly, the global attributes specified in the VIP file by the user and all of the entries for the VIP parameters when the retrieval was performed are also included.